Lab7

Accumulator A

Verilog Code

```
`default_nettype none

module Accumulator_A #
(
    parameter N = 4
)
(
    input MainClock,
    input ClearA,
    input LatchA,
    input EnableA,
    input [N-1:0] A,
    output [N-1:0] IB_BUS,
    output reg [N-1:0] AluA
);

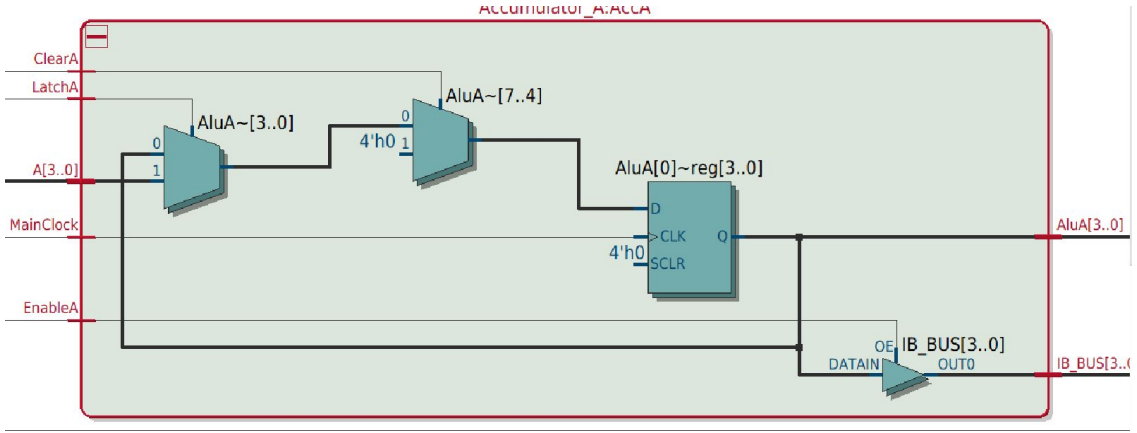
    // Sequential Logic
    always @(posedge MainClock) begin
        if(ClearA)
            AluA <= {N{1'b0}};
        else begin
            if(LatchA)
                AluA <= A;
        end
    end

    // Combinational Logic
    assign IB_BUS = (EnableA) ? AluA : {N{1'bz}};

endmodule

`default_nettype wire
```

RTL Viewer

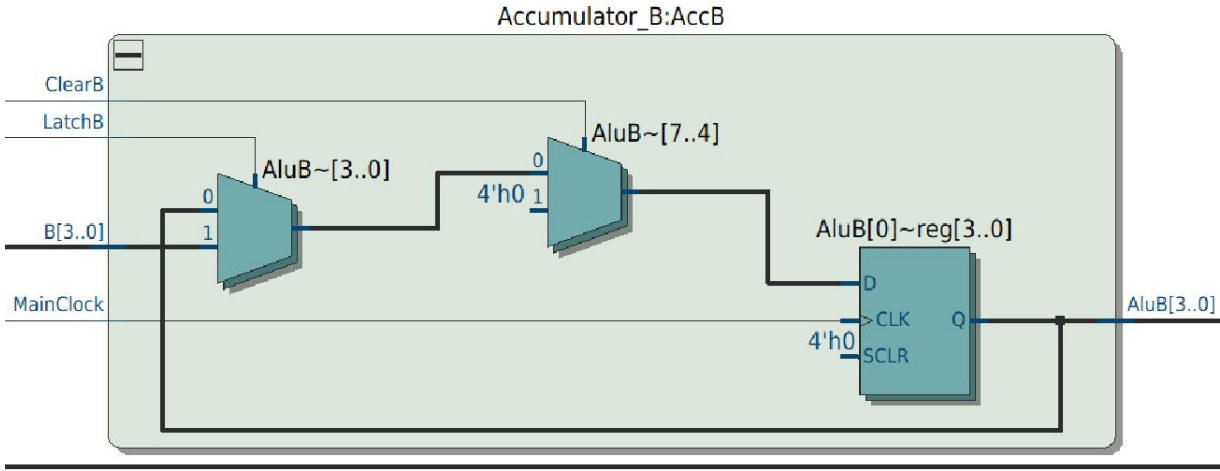


Accumulator B

Verilog Code

```
1
2   `default_nettype none
3
4   module Accumulator_B #
5   (
6      parameter N = 4
7   )
8   (
9      input MainClock,
10     input ClearB,
11     input LatchB,
12     input [N-1:0] B,
13     output reg [N-1:0] AluB
14  );
15
16     always @(posedge MainClock) begin
17        if (ClearB) begin
18           AluB <= {N{1'b0}};
19        end
20        else begin
21           if (LatchB) begin
22              AluB <= B;
23           end
24        end
25     end
26
27  endmodule
28
29  `default_nettype wire
30
```

RTL Viewer



Arithmetic Logic Unit

Verilog Code

```
`default_nettype none

module Arithmetic_Unit #
(
   parameter N = 4
)
(
   input EnableALU,
   input AddSub,
   input [N-1:0] A,
   input [N-1:0] B,
   output reg Carry,
   output reg [N-1:0] IB_ALU
);

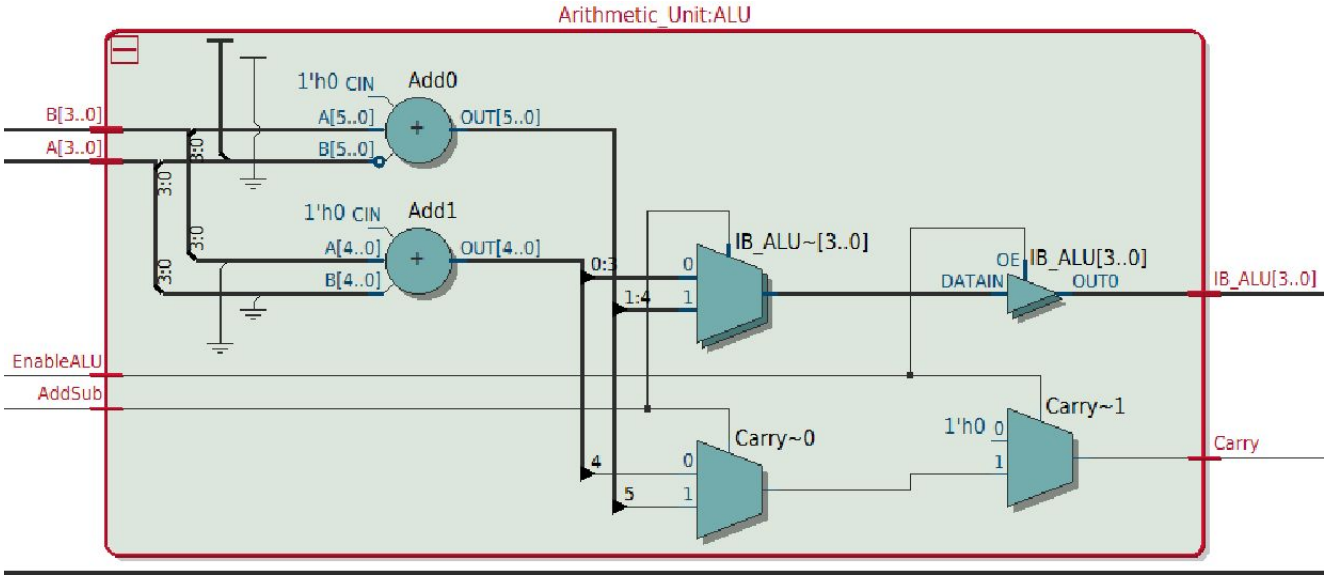
   always @(*) begin
      if (EnableALU) begin
         if (AddSub) begin
            {Carry, IB_ALU} = A - B;
         end
         else begin
            {Carry, IB_ALU} = A + B;
         end
      end
      else begin
         IB_ALU = {N{1'bz}};
         Carry = 1'b0;
      end
   end

endmodule

`default_nettype wire
```

RTL Viewer

Changed order of what gets subtracted to what, same for addition



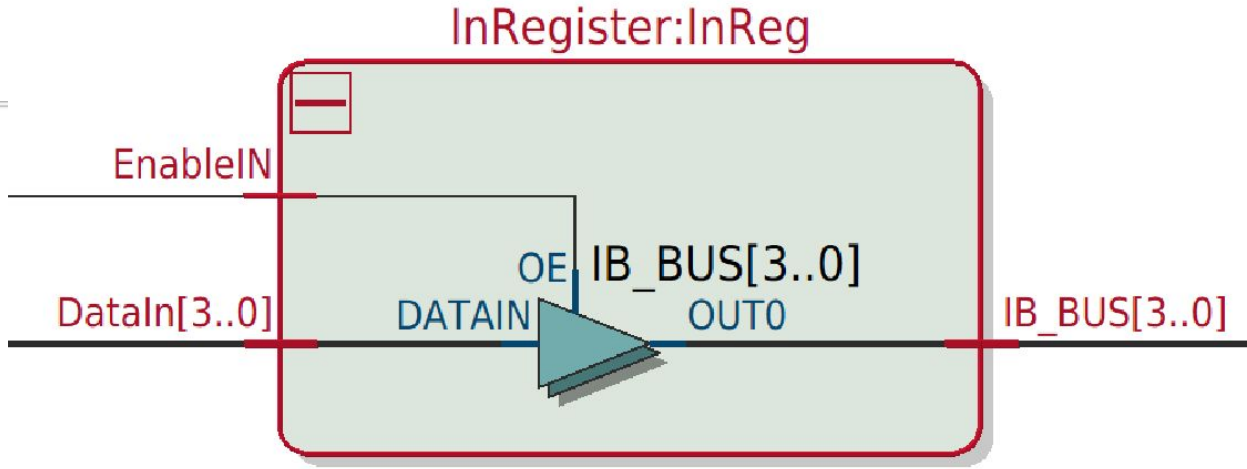
Prevented most negative number cases

Input Register

Verilog Code

```
1
2   `default_nettype none
3
4   module InRegister #
5   (
6       parameter N = 4
7   )
8   (
9       input EnableIN,
10      input [N-1:0] DataIn,
11      output [N-1:0] IB_BUS
12  );
13
14      assign IB_BUS = (EnableIN) ? DataIn : {N{1'bz}};
15
16  endmodule
17
18  `default_nettype wire
```

RTL Viewer

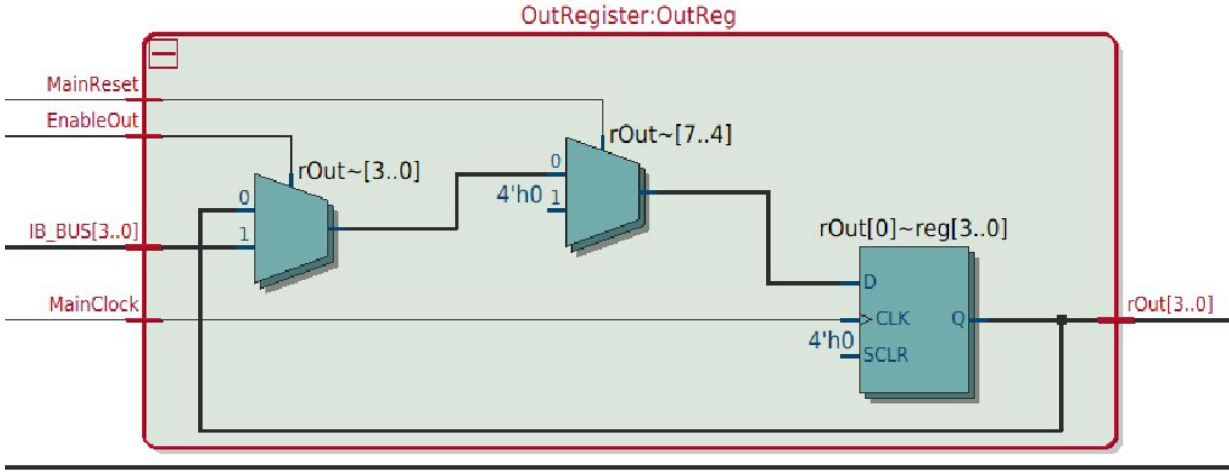


Out Register

Verilog Code

```
1
2   `default_nettype none
3
4   module OutRegister #
5   (
6       parameter N = 4
7   )
8   (
9       input MainClock,
10      input MainReset,
11      input EnableOut,
12      input [N-1:0] IB_BUS,
13      output reg [N-1:0] rOut
14  );
15
16      always @(posedge MainClock) begin
17          if (MainReset) begin
18              rOut <= {N{1'b0}};
19          end else begin
20              if (EnableOut) begin
21                  rOut <= IB_BUS;
22              end
23          end
24      end
25
26  endmodule
27
28  `default_nettype wire
```

RTL Viewer

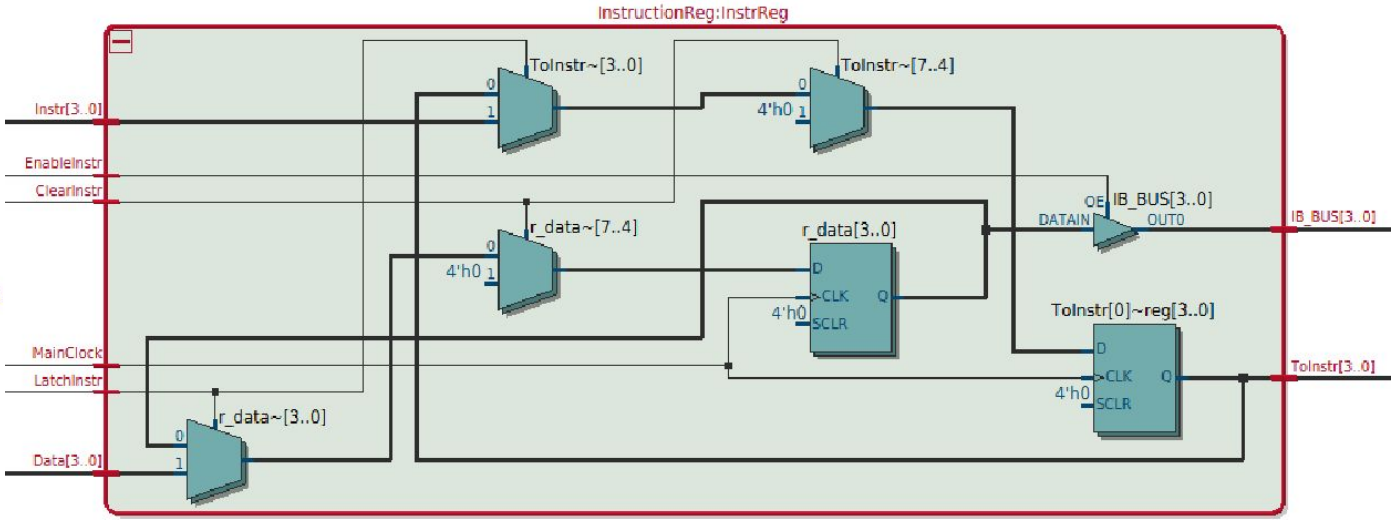


Instruction Register

Verilog Code

```
1
2   `default_nettype none
3
4   module InstructionReg #
5   (
6       parameter N = 4
7   )
8   (
9       input MainClock,
10      input ClearInstr,
11      input LatchInstr,
12      input EnableInstr,
13      input [N-1:0] Data,
14      input [N-1:0] Instr,
15      output reg [N-1:0] ToInstr,
16      output [N-1:0] IB_BUS
17  );
18
19      reg [N-1:0] r_data;
20
21      // sequential
22      always @(posedge MainClock) begin
23          if (ClearInstr) begin
24              ToInstr <= {N{1'b0}};
25              r_data <= {N{1'b0}};
26          end else begin
27              if (LatchInstr) begin
28                  ToInstr <= Instr;
29                  r_data <= Data;
30              end
31          end
32      end
33
34      //combinational
35      assign IB_BUS = (EnableInstr) ? r_data : {N{1'bz}};
36  endmodule
37
38  `default_nettype wire
39
```

RTL Viewer



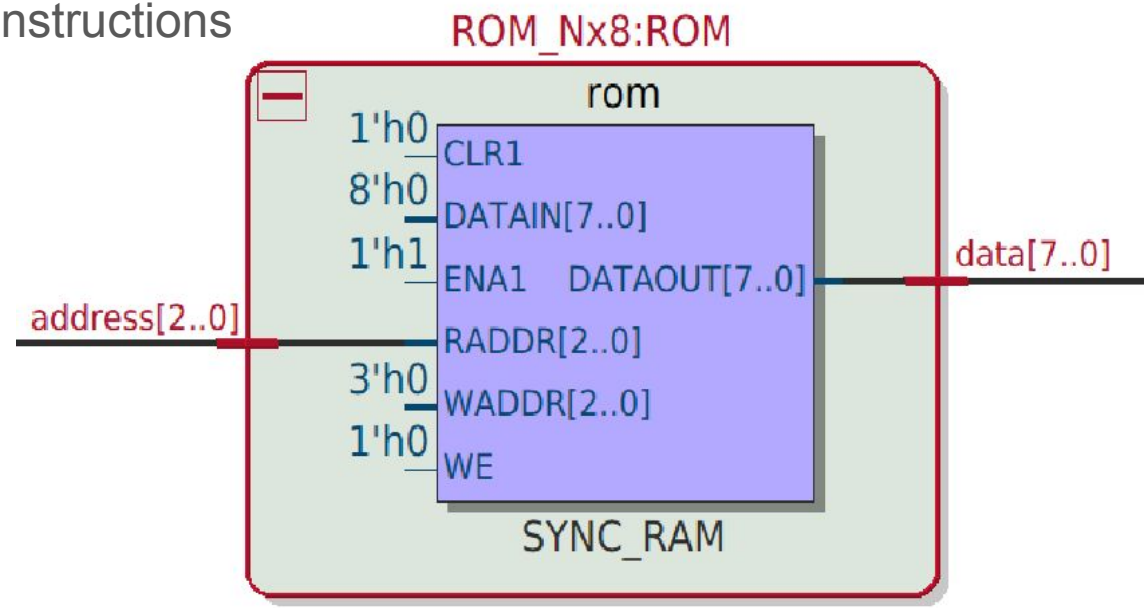
Program Memory (ROM)

Verilog Code

```
1
2   `default_nettype none
3
4   module ROM_Nx8 #
5   (
6       parameter N = 16
7   )
8   (
9       input [$clog2(N)-1:0] address,
10      output reg [7:0] data
11  );
12
13      (* ramstyle = "logic" *) reg [7:0] rom [0:N-1];
14
15      // Initialize memory -> your asm instructions
16      initial begin
17  rom[0] = 8'h54;  // LDA 4: A = 4
18  rom[1] = 8'h13;  // ADD 3: A = 7
19  rom[2] = 8'h30;  // OUT: display 7
20
21  rom[3] = 8'h40;  // IN: A = value from inregister
22  rom[4] = 8'h12;  // ADD 2: A = input value + 2
23  rom[5] = 8'h21;  // SUB 1: A = input value - 1
24  rom[6] = 8'h30;  // OUT: display input value - 1
25
26  rom[7] = 8'h53;  // LDA 3: A = 3
27  rom[8] = 8'h14;  // ADD 4: A = 7
28  rom[9] = 8'h30;  // OUT: display 7
29        // maximum 16 instructions
30      end
31
32      // Combinational Logic (async)
33      always @(*) begin
34          data = rom[address];
35      end
36
37  endmodule
38
39  `default_nettype wire
40
```

RTL Viewer

Changed bit to allow for max of 16 instructions

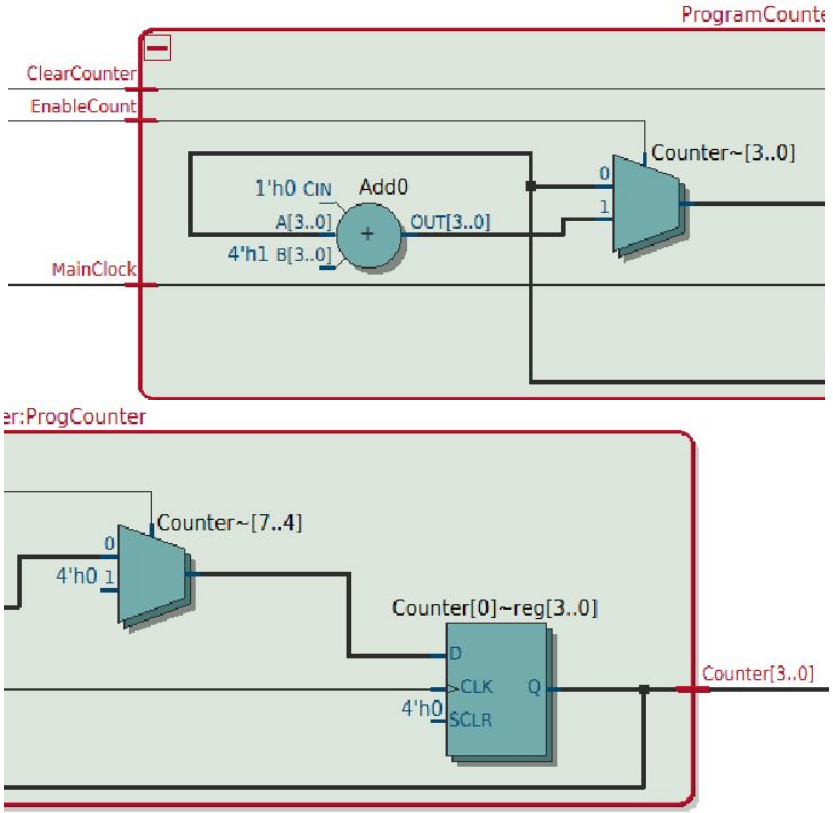


Program Counter

Verilog Code

```
1
2   `default_nettype none
3
4   module ProgramCounter #
5   (
6       parameter N = 4
7   )
8   (
9       input MainClock,
10      input EnableCount,
11      input ClearCounter,
12      output reg [N-1:0] Counter
13  );
14
15      always @(posedge MainClock) begin
16          if (ClearCounter) begin
17              Counter = {N{1'b0}};
18          end else begin
19              if (EnableCount) begin
20                  Counter <= Counter + 1'b1;
21              end
22          end
23      end
24  endmodule
25
26  `default_nettype wire
27
```

RTL Viewer



FSM - Controller

Verilog Code

```
1
2   `default_nettype none
3
4   module FSM_MicroInstr #
5   (
6       parameter N = 4
7   )
8   (
9       input clk,
10      input reset,
11      input [N-1:0] IB_BUS,
12
13      output reg LatchA,
14      output reg EnableA,
15      output reg LatchB,
16      output reg EnableALU,
17      output reg AddSub,
18      output reg EnableIN,
19      output reg EnableOut,
20      output reg LoadInstr,
21      output reg EnableInstr,
22      input [N-1:0]  ToInstr,
23      output reg EnableCount
24  );
25
26      reg [2:0] state, next_state;
27
28      // States
29      localparam [2:0] IDLE = 3'd0, PHASE_1 = 3'd1, PHASE_2 = 3'd2, PHASE_3 = 3'd3, PHASE_4 = 3'd4;
30
31      // 1. State Register (sequential)
32      always @(posedge clk) begin
33          if(reset)
34              state <= IDLE;
35          else
36              state <= next_state;
```

```
37      end
38
39      // 2. Next-State Logic (combinational)
40      always @(*) begin
41
42          next_state = state; // default: hold state
43
44          case(state)
45
46              IDLE:
47                  begin
48                      // Move to FETCH
49                      next_state = PHASE_1;
50                  end
51
52              // FETCH Instruction
53              PHASE_1:
54                  begin
55                      // Move to DECODE
56                      next_state = PHASE_2;
57                  end
58
59              // DECODE Instruction
60              PHASE_2:
61                  begin
62                      // Move to EXECUTE A
63                      next_state = PHASE_3;
64                  end
65
66              // EXECUTE A Instruction
67              PHASE_3:
68                  begin
69                      // Move to EXECUTE B
70                      next_state = PHASE_4;
71                  end
72
```

Verilog Code

```
73          // EXECUTE B Instruction
74          PHASE_4:
75             begin
76                // Move to FETCH
77                next_state = PHASE_1;
78             end
79
80          default: ;// None
81
82       endcase
83    end
84
85    // 3. Output Logic (combinational)
86
87    always @(*) begin
88
89       // default all signals are zero
90       LoadInstr   = 1'b0;
91       EnableInstr = 1'b0;
92       LatchB      = 1'b0;
93       LatchA      = 1'b0;
94       EnableALU   = 1'b0;
95       EnableCount = 1'b0;
96       AddSub      = 1'b0;
97       EnableIN    = 1'b0;
98       EnableA     = 1'b0;
99       EnableOut   = 1'b0;
100
101      case(state)
102
103         IDLE:
104            begin
105               // Initialize things if needed
106            end
107
108         // FETCH Instruction
```

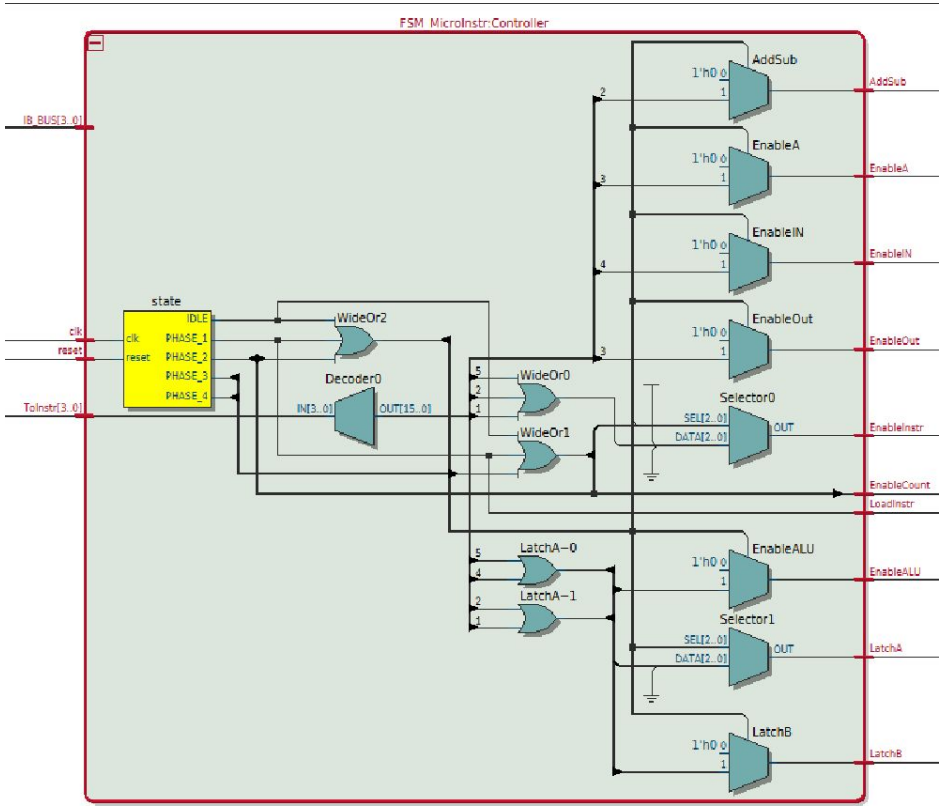
```
108   // FETCH Instruction
109   PHASE_1:
110      begin
111         // same sequence for all asm instructions
112         LoadInstr = 1'b1;
113      end
114
115   // DECODE Instruction
116   PHASE_2:
117      begin
118         // same sequence for all asm instructions
119         EnableCount = 1'b1;
120         EnableInstr = 1'b1;
121      end
122
123   // EXECUTE Instruction
124   PHASE_3:
125      begin
126
127         case(ToInstr)
128
129            4'b0000: // NOP
130               begin
131                  // do nothing
132               end
133
134            4'b0001: // ADD
135               begin
136                  LatchB      = 1'b1;
137                  EnableInstr = 1'b1;
138               end
139
140            4'b0010: // SUB
141               begin
142                  LatchB      = 1'b1;
143                  EnableInstr = 1'b1;
```

```
143                  EnableInstr = 1'b1;
144               end
145
146            4'b0011: // OUT
147               begin
148                  EnableOut   = 1'b1;
149                  EnableA     = 1'b1;
150               end
151
152            4'b0100: // IN
153               begin
154                  LatchA   = 1'b1;
155                  EnableIN = 1'b1;
156               end
157
158            4'b0101: // LDA
159               begin
160                  LatchA      = 1'b1;
161                  EnableInstr = 1'b1;
162               end
163
164         endcase
165
166      end
167
168   // EXECUTE Instruction
169   PHASE_4:
170      begin
171
172         case(ToInstr)
173
174            4'b0000: // NOP
175               begin
176                  // do nothing
177               end
178
179
```

Verilog Code

```
178
179                     4'b0001: // ADD
180                        begin
181                           LatchA      = 1'b1;
182                           EnableALU   = 1'b1;
183                        end
184
185                     4'b0010: // SUB
186                        begin
187                           LatchA      = 1'b1;
188                           EnableALU   = 1'b1;
189                           AddSub      = 1'b1;
190                        end
191
192                     4'b0011: // OUT
193                        begin
194                           // do nothing
195                        end
196
197                     4'b0100: // IN
198                        begin
199                           // do nothing
200                        end
201
202                     4'b0101: // LDA
203                        begin
204                           // do nothing
205                        end
206
207                  endcase
208
209               end
210
211            endcase
212         end
213
214   endmodule
215
216   `default_nettype wire
217
```

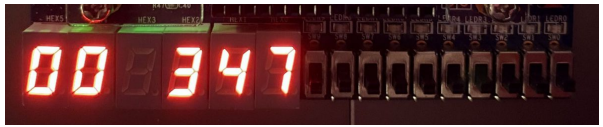
RTL Viewer



Board Implementation



Loads value 4



Adds 3 to previous



Displays output



Loads inregister and adds 2



Subtracts 1 from previous



Displays output



Loads value 3



Adds 4 to previous



Displays output